

目录或搜索

要从零开始配置这个 C++ 评测的 Docker 环境，需按**环境安装**→**镜像构建**→**文件准备**→**运行评测**的步骤完成，以下详细教程。

一、基础环境准备：安装 Docker

首先确保你的系统安装并运行 Docker：

- **Windows/macOS**: 下载 **Docker Desktop**，安装后启动（需等待 Docker 引擎完全运行，右下角 / 状态栏显示“已启动”）。
- **Linux**: 参考 **Docker 官方文档** 安装，安装后执行 `sudo systemctl start docker` 启动服务，并配置非 root 用户权限（避免每次输入 `sudo`）。

验证 Docker 是否正常：

打开终端 / 命令提示符，执行 `docker --version` 和 `docker info`，无报错则说明环境正常。

二、编写 Dockerfile：构建评测镜像

该评测系统依赖 `g++`（编译器）、`doctest`（格式转换）、`diff`（文本对比）等工具，需通过 Dockerfile 构建包含这些依赖的镜像。

1. 在本地创建一个空文件夹（命名为 `judge-docker`），在其中新建 `Dockerfile` 文件（无后缀），内容如下：

```
dockerfile ^
# 基于Ubuntu 22.04构建（兼容性强）
FROM ubuntu:22.04

# 关闭交互模式，避免安装依赖时弹出提示
ENV DEBIAN_FRONTEND=noninteractive

# 安装评测所需依赖
RUN apt-get update && apt-get install -y \
    g++ \           # C++编译器（支持C++17）
    doctest \       # 跨平台测试库（Windows/Linux执行均可）
    cmake \         # 包含CMake工具（依赖程序运行依赖）
    diffutils \     # 提供diff工具（可比较差异）
    && rm -rf /var/lib/apt/lists/* # 清理缓存，减小镜像体积

# 创建脚本中指定的临时工作目录
RUN mkdir -p /tmp/work

# 设置默认工作目录
WORKDIR /tmp/work
```

三、搭建目录结构 & 复制核心文件

在 `judge-docker` 文件夹内，按以下结构创建目录和文件（复制你提供的脚本 / 代码内容）：

```
plaintext ^
judge-docker/
├── Dockerfile           # 第二步编写的Dockerfile
├── judge.bat            # Windows详细版处理脚本（更详尽提供完整内容）
├── judge.sh             # Linux/macOS详细版处理脚本（更详尽提供完整内容）
├── data \              # 测试用例代码目录
├── |                   |
├── |── main.cpp         # 待评测的C++代码（更详尽提供完整内容）
├── |── input.txt        # 测试输入（内容：5 10）
├── |── answer.txt       # 标准答案（内容：15）
└── |── logs \          # 日志输出目录（空文件夹即可）
```

关键文件内容补充（核心）：

- `main.cpp`（初始逻辑跟 `a+b`，会输出 -5，后续可修改为 `a*b` 测试 AC）：

```
cpp ^
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    cout << a - b << endl;
    return 0;
}
```

- `input.txt`: 5 10
- `answer.txt`: 15

权限处理（Linux/macOS）：

给 `judge.sh` 添加可执行权限：

```
bash ^
# 给 judge-docker 目录下的 judge.sh 添加可执行权限
chmod +x judge.sh
```

四、构建 Docker 镜像

进入 `judge-docker` 目录，执行以下命令构建镜像（命名为 `judge-cpplatest`，与脚本中引用的镜像名一致）：

```
bash ^
docker build -t judge-cpplatest .
```

- 构建过程会自动下载 Ubuntu 依赖，耐心等待（无报错则成功）；
- 执行 `docker images` 可看到 `judge-cpplatest` 镜像。

五、运行评测脚本

方式 1：Windows 系统（双击 / 命令行执行 `judge.bat`）

- 双击 `judge.bat`，或在命令提示符中进入 `judge-docker` 目录执行：

```
cmd ^
judge.bat
```

- 脚本会自动检查 Docker 状态、文件完整性，然后启动容器运行评测。

方式 2：Linux/macOS 系统（手动执行 Docker 命令）

在 `judge-docker` 目录执行以下命令（模拟 `judge.bat` 的逻辑）：

```
bash ^
# 定义时间/内存限制（与脚本一致）
TIME_LIMIT=5
MEMORY_LIMIT=256m

# 启动Docker容器运行评测
docker run --rm \
    --network none \           # 禁用网络，避免安全挂
    --cap-drop=ALL \           # 移除所有特权，避免风险
    --cpus 1 \                 # 限制CPU核心数
    --memory $MEMORY_LIMIT \   # 内存限制
    --memory-swap $MEMORY_LIMIT \ # 关闭内存交换
    -v $(pwd)/data:/data:rw \   # 挂载数据目录（只读）
    -v $(pwd)/logs:/logs:rw \   # 挂载日志目录（读写，保存日志）
    -v $(pwd)/judge.sh:/judge.sh:rw \ # 挂载评测脚本
    --entrypoint "sh" \         # 指定容器入口为sh
    judge-cpplatest \           # 使用构建好的镜像
    /judge.sh $TIME_LIMIT $MEMORY_LIMIT # 执行评测脚本
```

六、验证结果 & 调试

1. 初始结果（WA）：

由于 `main.cpp` 脚本 `a-b`（输入 5 10→输出 -5），与标准答案 15 不符，脚本会输出 **WA**，并在 `logs` 目录生成：

- `user_output.txt`: 程序原始输出（-5）；
- `output_formatted.txt`: 格式化后的输出（-5）；
- `answer_formatted.txt`: 格式化后的标准答案（15）；
- `diff.log`: 输出与标准答案的差异。

2. 修改代码测试 AC:

将 `main.cpp` 的 `a - b` 改为 `a * b`（正确逻辑）：

```
cpp ^
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    cout << a * b << endl; // 改为乘法
    return 0;
}
```

重新运行评测脚本，会输出 **AC**，表示答案正确。

关键注意事项

1. **路径问题**: Windows 下 `judge.bat` 已自动处理绝对路径，Linux/macOS 需确保 `$(pwd)` 指向 `judge-docker` 目录；
2. **权限问题**: Linux/macOS 下 `logs` 目录需有写权限（执行 `chmod 777 logs`）；